```
    sendBuf.compact();    #B

    while (!sslEngine.isInboundDone()) {    #C
        channel.receive(recvBuf);    #C
        sslEngine.unwrap(recvBuf.flip(), appData);    #C
        recvBuf.compact();    #C
    }
    sslEngine.closeInbound();    #D
    channel.close();    #D
}
```

#A Indicate that no further outbound application packets will be sent
#B Call wrap() to generate the close-notify message and send it to the server
#C Wait until a close-notify is received from the server
#D Indicate that the inbound side is now done too and close the UDP channel

IMPLEMENTING A DTLS SERVER

Initializing a `SSLContext` for a server is similar to the client, except in this case you use a `KeyManagerFactory` to supply the server's certificate and private key. Because you're not using client certificate authentication, you can leave the `TrustManager` array as `null`. Listing 12.5 shows the code for creating a server-side DTLS context. Create a new file named DtlsServer.java next to the client and type in the contents of the listing.

Listing 12.5 The server SSLContext

```
package com.manning.apisecurityinaction;
```